

Performance Testing

Date	04 July 2026
Team ID	
Project Name	Credit Card Approval Prediction
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Manual Testing
Type of Testing	Load Testing
Target Module / API	Credit Card Approval Prediction
Test Environment	Local System
Test Date	

Step 2: Test Scenarios

S. No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Single user submits a Credit Card Approval Prediction	1	30	Prediction result is (Approved or rejected) displayed correctly.

2	Multiple users submit prediction requests simultaneously	10	60	Application processes all successfully without errors.
3	Continuous prediction requests	20	120	Application remains stable and provides accurate predictions.



Step 3: Performance Test Results

S. No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Average)	< 2 seconds	1.3 seconds	Pass	Fast response for prediction requests.
2	Response Time (Max)	< 5 seconds	2.9 seconds	Pass	Maximum response time remained acceptable limits
3	Throughput (Req/sec)	10 Req/sec	11 Req/sec	Pass	Application handled requests efficiently.
4	Error Rate	< 1%	0%	Pass	No errors observed during testing.
5	CPU Utilization	< 80%	46%	Pass	CPU usage remained stable throughout testing.
6	Memory Utilization	< 80%	54%	Pass	Memory usage was within acceptable limits.

Step 4: Observations & Analysis

Key Findings:

The **Credit Card Approval Prediction** system performed efficiently during testing. The average response time remained below 2 seconds, CPU and memory utilization stayed within acceptable limits, and all prediction requests were processed successfully without failures. The application maintained consistent performance under both normal and moderate workloads..

Bottlenecks Identified:

No significant performance bottlenecks were identified during testing. The application remained stable while processing multiple prediction requests, with no noticeable delays or errors.

Optimization steps taken

- ☐ Implemented input validation to prevent invalid application data.
- ☐ Optimized data preprocessing to reduce unnecessary computations.
- ☐ Loaded the trained machine learning model once during application startup to minimize prediction latency.
- ☐ Improved backend request handling for faster response times and better resource utilization.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

